BusyLean Contender 7

Autonomous Critical-Point Reflection over a Self-Generated W-Tree Index

Research proposal prepared for the BusyLean project

October 2026

Abstract

The BusyLean repository asks for closed, constructive Lean definitions of very large natural numbers together with kernel-checked proofs that each new entry exceeds the preceding one. The current entry, Contender 6, applies forest reflection over a finite-type W-language along an index that joins every $W\mathbb{N}$-tree definable in that same language. This paper proposes Contender 7, called *autonomous critical-point reflection*.

The construction first embeds the Contender 6 index into a transparent Brouwer-tree syntax. It then closes that seed under ordinal-style addition, multiplication, exponentiation, fixed-point enumeration, a finite hierarchy of critical functions, a diagonal common-critical-point operation, and forty-two levels of higher derivatives. The resulting autonomous payloads are inserted into a guarded branch tower. Every successor branch contains the entire preceding branch as child 0 and the next autonomous payload as child 1. At input 42, the outer reflection root exposes branch 42. Lean therefore proves a chain of forty-three strict reflection steps above the complete Contender 6 index, as well as a separate strict lower bound by the level-41 autonomous payload.

The exact submitted definition is a closed term `BusyLean.contender_7 : Nat`. All recursion is structural; no `noncomputable`, `partial`, `unsafe`, custom axiom, `Classical.choice`, previous contender constant, or evaluation shortcut is used. The formal theorem is

`BusyLean.contender_6_lt_contender_7 : contender_6 < contender_7.`

The theorem is independent of any informal ordinal calibration. The latter is presented separately and explicitly labelled as heuristic.

Contents

1 Problem and result **2**
- 1.1 The BusyLean setting . . . 2
- 1.2 The proposed entry . . . 3

2 The inherited reflection engine **4**
- 2.1 Oracle families and one-step reflection . . . 4
- 2.2 Forest reflection . . . 4
- 2.3 Contender 6 as the seed schedule . . . 4

3 A transparent autonomous tree language **4**
- 3.1 Raw syntax . . . 4
- 3.2 Arithmetic constructors . . . 5
- 3.3 Fixed-point enumeration . . . 5
- 3.4 Critical hierarchy and autonomous closure . . . 6

4 Embedding the old index and realizing new schedules **6**
4.1 Encoding and realization . . . 6
4.2 Why the seed is not merely an external notation . . . 7

5 The guarded branch tower **7**
5.1 Definition . . . 7
5.2 Why the guard is useful . . . 7

6 Formal comparison in Lean **8**
6.1 One guarded step . . . 8
6.2 The forty-two-step chain . . . 8
6.3 The outer step and final theorem . . . 8
6.4 Payload inclusion theorem . . . 9

7 Computability and constructive status **9**
7.1 Why the number is computable in principle . . . 9
7.2 Why non-evaluation is not noncomputability . . . 9
7.3 Dependency discipline . . . 9

8 Growth analysis: formal facts versus heuristic calibration **10**
8.1 What is formally established . . . 10
8.2 Intended ordinal reading . . . 10
8.3 Why this still constitutes a significant advance . . . 10

9 Verification protocol **11**

10 Limitations and open obligations **11**

11 Further research questions **12**

12 Integration notes **12**

13 Conclusion **13**

A Principal theorem inventory **13**

B Reproduction commands **14**

C Bibliographic note **14**

1 Problem and result

1.1 The BusyLean setting

BusyLean is a Lean 4 continuation of the "name the biggest number" style of formal competition. A legitimate entry is not merely a suggestive notation: it is a closed Lean definition of type `Nat`, constructive and computable in principle, accompanied by a kernel-checked strict comparison with the incumbent. The repository's engineering rules also exclude the usual escape hatches: no classical choice in computational dependencies, no custom axioms, no `sorry`, no `native_decide` certificate, and no definition that simply invokes the previous named contender.

The incumbent in the repository at the time of this work is Contender 6:

```
def contender_6 : Nat := stage p5Family index 42
```

Here `stage` is forest reflection over a W-type object language, and `index` is assembled from every $W\mathbb{N}$-tree denoted by an enumerated program of that language. Its comparison with the exact Lean port of Contender 5 is already proved in the repository.

1.2 The proposed entry

The new definition is:

```
def contender7Base : Autonomous.OTree :=
  Autonomous.encode index

def contender7RawBranch (k : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper k
      (Autonomous.OTree.succ contender7Base))

def contender7Branch : Nat -> WTree Nat
  | 0 => index
  | k + 1 =>
      .node (fun i =>
        match i with
        | 0 => contender7Branch k
        | _ + 1 => contender7RawBranch k)

def contender7Index : WTree Nat :=
  .node contender7Branch

def contender7Family (n : Nat) : Nat :=
  stage p5Family contender7Index n

def contender_7 : Nat := contender7Family 42
```

The construction has two deliberately separate margins.

1. **A fully formal guard margin.** Branch 0 is exactly the complete Contender 6 index. Branch $k+1$ contains branch k as child 0. Thus, at input 42, Lean proves

$$\mathsf{stage}(B_0, 42) < \mathsf{stage}(B_1, 42) < \cdots < \mathsf{stage}(B_{42}, 42) < \mathsf{stage}(\mathsf{root}, 42).$$

   This is a chain of forty-three strict reflection steps above the old index.

2. **An autonomous ordinal-style payload.** Branch $k+1$ also contains as child 1 a realization of the k-th higher derivative of a common-critical-point closure above the encoded old index. Hence branch 42 formally exceeds the level-41 payload, and the final root exceeds branch 42.

The required theorem is proved through the long guard chain rather than by the shorter direct observation that branch 0 is a child of the root. This records a meaningful structural margin in the proof term itself.

2 The inherited reflection engine

This proposal deliberately reuses the trusted reflection mechanism already present in BusyLean. Only the index schedule is strengthened.

2.1 Oracle families and one-step reflection

A family is a binary natural-valued oracle:

$$\mathsf{Family} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}.$$

For a family F, the existing repository defines

$$\mathsf{reflect}(F, n) = n + 1 + \max\left(\max_{i \le n} F(i, n), \max_{t \in \mathsf{allTerms}(n)} \mathsf{evalWith}(F, t)\right).$$

The implementation uses primitive list maxima and the enumerator of the finite-type W-language. Therefore every visible oracle value and every visible program value is strictly smaller than $\mathsf{reflect}(F, n)$.

2.2 Forest reflection

For $a : \mathsf{WTree}(\mathbb{N})$, the inherited function stageis defined structurally:

```
def stage (seed : Family) : WTree Nat -> Nat -> Nat
  | .leaf   => reflect seed
  | .node f => reflect (fun i => stage seed (f i))
```

At a node, the oracle family is precisely the family of recursively computed child stages. The key theorem is:

Theorem 1 (Visible-child domination)**.** *For every seed family S, child function $f : \mathbb{N} \to \mathsf{WTree}(\mathbb{N})$, input n, and index $i \le n$,*

$$\mathsf{stage}(S, f(i), n) < \mathsf{stage}(S, \mathsf{node}(f), n).$$

This theorem is the only fact required for the formal comparison. No monotonicity of reflection in the oracle is used, because such monotonicity is false in general: an object-language program may inspect an oracle value and branch in a way that reverses a pointwise comparison.

2.3 Contender 6 as the seed schedule

Contender 6's `index` is a node whose k-th child joins, in enumeration order, all trees denoted by programs of depth at most k. Thus the old schedule already reflects along the object language's own definable trees. In this proposal that entire schedule is treated as one indivisible base object. It is not re-enumerated, approximated, or replaced by a weaker external notation.

3 A transparent autonomous tree language

3.1 Raw syntax

We introduce a small Brouwer-tree syntax:

```
inductive OTree : Type where
  | zero  : OTree
  | succ  : OTree -> OTree
  | limit : (Nat -> OTree) -> OTree
```

The type is inductive and strictly positive. Every inhabitant is a finite Lean value that may contain functions producing further finite values. Structural recursion on this W-type is accepted by Lean's termination checker.

One can assign an external ordinal reading by

$$\begin{aligned} |\mathsf{zero}| &= 0, \\ |\mathsf{succ}(a)| &= |a| + 1, \\ |\mathsf{limit}(f)| &= \sup_{n<\omega}(|f(n)| + 1). \end{aligned}$$

This interpretation is explanatory only; the submitted Lean comparison does not depend on an ordinal quotient, ordinal equality, or a formalization of countable ordinals.

3.2 Arithmetic constructors

The source defines syntax-level addition, multiplication, and exponentiation by structural recursion in the right argument:

```
def add (a : OTree) : OTree -> OTree
  | .zero    => a
  | .succ b  => .succ (add a b)
  | .limit f => .limit (fun n => add a (f n))

def mul (a : OTree) : OTree -> OTree
  | .zero    => .zero
  | .succ b  => add (mul a b) a
  | .limit f => .limit (fun n => mul a (f n))

def pow (a : OTree) : OTree -> OTree
  | .zero    => .succ .zero
  | .succ b  => mul (pow a b) a
  | .limit f => .limit (fun n => pow a (f n))
```

With the usual cofinality hypotheses on limit sequences, these are the familiar recursive clauses for ordinal addition, multiplication, and exponentiation. The code intentionally avoids asserting quotient-level algebraic laws that have not been formalized.

Let `omega` be the limit of finite successor chains, and define `omegaPow a := pow omega a`. This supplies the level-zero normal-function syntax $\alpha \mapsto \omega^{\alpha}$.

3.3 Fixed-point enumeration

For a syntax transformer $F : \mathsf{OTree} \to \mathsf{OTree}$, define

```
def derivative (f : OTree -> OTree) (a : OTree) : OTree :=
  .limit (fun n => iterate f (n + 1) (.succ a))
```

The mathematical motivation is Veblen's fixed-point construction.

Proposition 2 (Classical fixed-point calculation)**.** *Let F be a normal ordinal function, and let a be an ordinal. Put $x_0 = a + 1$, $x_{n+1} = F(x_n)$, and $\lambda = \sup_{n<\omega} x_n$. Then $F(\lambda) = \lambda$, and λ is the least fixed point of F not below x_0.*

Proof. Continuity gives $F(\lambda) = F(\sup_n x_n) = \sup_n F(x_n) = \sup_n x_{n+1} = \lambda$. If $y \geq x_0$ and $F(y) = y$, monotonicity gives $x_n \leq y$ for every n by induction, hence $\lambda \leq y$. □

The Lean term `derivative` is total without assuming normality. Normality is needed only for the intended ordinal interpretation.

3.4 Critical hierarchy and autonomous closure

Define finite critical levels by

$$C_0(a) = \omega^a,$$
$$C_{k+1}(a) = C_k'(a),$$

where the prime denotes the preceding fixed-point enumerator. In Lean:

```
def critical : Nat -> OTree -> OTree
  | 0, a     => omegaPow a
  | k + 1, a => derivative (fun b => critical k b) a
```

Next take a diagonal common-critical-point closure:

```
def autonomous (a : OTree) : OTree :=
  .limit (fun k => critical (k + 1) (.succ a))
```

The word "autonomous" refers to the classical proof-theoretic idea of closing an ordinal notation under successively stronger critical operations rather than choosing one fixed external level. The construction is Veblen-inspired and Gamma-like, but this paper intentionally does *not* identify its exact quotient-level order type with Γ_0 or any other named ordinal. Such an identity would require a formal semantic development and a careful analysis of fundamental sequences.

Finally, iterate the fixed-point operation on the autonomous transformer:

```
def hyper : Nat -> OTree -> OTree
  | 0, a     => autonomous a
  | k + 1, a => derivative (fun b => hyper k b) a
```

At input 42, the proposed contender exposes `hyper 41`. This is a finite closed definition, not an appeal to an unbounded search or a noncomputable "least undefinable" object.

4 Embedding the old index and realizing new schedules

4.1 Encoding and realization

The old reflection engine expects `WTree Nat`, whereas the autonomous operators act on `OTree`. Two transparent structural maps connect them:

```
def encode : WTree Nat -> OTree
  | .leaf   => .zero
  | .node f => .limit (fun n => encode (f n))

def realize : OTree -> WTree Nat
  | .zero   => .leaf
```

```
| .succ a =>
    .node (fun n =>
      match n with
      | 0     => realize a
      | _ + 1 => .leaf)
| .limit f => .node (fun n => realize (f n))
```

Under the rank convention $\mathrm{rk}(\mathsf{leaf}) = 0$ and $\mathrm{rk}(\mathsf{node}(f)) = \sup_n(\mathrm{rk}(f(n)) + 1)$, `realize` has the expected behavior: zero becomes rank 0, successor adds a visible predecessor child, and limit preserves a countable supremum. Again, rank preservation is an intended semantic invariant, not an assumption used by the strict numerical comparison.

4.2 Why the seed is not merely an external notation

The base term is

$$B = \texttt{Autonomous.encode BusyLean.index}.$$

Thus the autonomous hierarchy starts *above the actual self-generated Contender 6 schedule*. This matters. A fixed Veblen notation built from zero could remain far below the proof-theoretic strength already encoded by the W-language. Starting from the old index instead applies the critical-point operations to that stronger base rather than attempting to catch it from below.

The level-k payload is

$$R_k = \mathsf{realize}\big(\mathsf{hyper}_k(\mathsf{succ}(B))\big).$$

The successor ensures that the first iteration begins strictly above the base under the intended ordinal reading.

5 The guarded branch tower

5.1 Definition

Let $B_0 = \mathsf{index}$. Recursively define

$$B_{k+1} = \mathsf{node}(g_k), \qquad g_k(0) = B_k, \qquad g_k(i+1) = R_k.$$

This is exactly `contender7Branch`. The final index is

$$J_7 = \mathsf{node}(k \mapsto B_k).$$

At input n, the reflection root is guaranteed to see every branch B_i with $i \le n$. Therefore input 42 sees B_{42}.

5.2 Why the guard is useful

The autonomous payload is intended to be much stronger than the old index, but a proof of that statement at the level of ordinal semantics would be a separate formalization project. The guard tower isolates the concerns:

- the exact strict comparison with Contender 6 follows from the already trusted visible-child theorem;
- the autonomous payload can be studied, strengthened, or replaced without endangering that comparison;

- the proof term explicitly records forty-two strict internal stages rather than hiding the old index in a single opaque maximum;
- child 1 witnesses that the large payload is genuinely present in the computed reflection family, not just discussed in comments.

The repetition of R_k at every positive child index is intentional. At any input $n \geq 1$, the payload is visible. It also avoids an auxiliary default branch and keeps the definitional reductions at indices 0 and 1 transparent.

6 Formal comparison in Lean

6.1 One guarded step

The repository theorem `stage_child_lt` immediately gives:

```
theorem contender7_branch_step_lt (k n : Nat) :
    stage p5Family (contender7Branch k) n <
      stage p5Family (contender7Branch (k + 1)) n := by
  -- child 0 of branch (k+1) is branch k
  exact stage_child_lt p5Family childFamily
    (i := 0) (n := n) (Nat.zero_le n)
```

The submitted source spells out `childFamily` as the defining match, so there is no extensionality argument or hidden rewrite.

Similarly, for $n \geq 1$, child 1 yields

$$\mathsf{stage}(R_k, n) < \mathsf{stage}(B_{k+1}, n).$$

6.2 The forty-two-step chain

Primitive recursion on k composes the strict inequalities:

```
theorem contender7_branch_zero_lt_succ :
    forall k n : Nat,
      stage p5Family (contender7Branch 0) n <
        stage p5Family (contender7Branch (k + 1)) n
  | 0, n => contender7_branch_step_lt 0 n
  | k + 1, n =>
      Nat.lt_trans (contender7_branch_zero_lt_succ k n)
        (contender7_branch_step_lt (k + 1) n)
```

Instantiating $k = 41$ and $n = 42$ gives

$$\mathsf{stage}(B_0, 42) < \mathsf{stage}(B_{42}, 42).$$

Because B_0 is definitionally the old `index`, the left side is exactly Contender 6.

6.3 The outer step and final theorem

Branch 42 is child 42 of the outer root and $42 \leq 42$, so

$$\mathsf{stage}(B_{42}, 42) < \mathsf{stage}(J_7, 42) = \mathsf{C}_7.$$

Transitivity yields:

Theorem 3 (Kernel-checked comparison)**.**

BusyLean.contender_6 < BusyLean.contender_7.

The Lean declaration is `BusyLean.contender_6_lt_contender_7`.

A second theorem composes this with the repository's existing comparison from Contender 5:

`BusyLean.contender_5 < BusyLean.contender_7`.

6.4 Payload inclusion theorem

Since R_{41} is child 1 of B_{42} and $1 \leq 42$, Lean also proves

$$\mathsf{stage}(R_{41}, 42) < \mathsf{stage}(B_{42}, 42) < \mathsf{C}_7.$$

The final declaration is `BusyLean.hyper41_lt_contender_7`. This separates the formal fact that the payload participates in the number from the informal claim about its ordinal magnitude.

7 Computability and constructive status

7.1 Why the number is computable in principle

Every definition used by Contender 7 is a total Lean function:

- `iterate`, `ofNat`, `critical`, `hyper`, and `contender7Branch` recurse structurally on `Nat`;
- `add`, `mul`, `pow`, and `realize` recurse structurally on `OTree`;
- `encode` and the inherited `stage` recurse structurally on well-founded W-trees;
- all maxima range over finite lists generated from a finite depth;
- the final term is closed and has type `Nat`.

Lean's acceptance of the definitions is itself a termination certificate for these recursive equations. In particular, the construction uses no halting oracle, Busy Beaver function, minimization over an undecidable predicate, or choice of an unspecified witness.

7.2 Why non-evaluation is not noncomputability

The numerical value is far beyond feasible evaluation. That is expected and does not conflict with computability. A closed structurally recursive program can have an astronomically long reduction sequence. The contest compares such terms intensionally, through proved inequalities, rather than attempting to print their decimal expansions.

7.3 Dependency discipline

The audit checks that `contender_7` does not depend on the named constants `contender_5`, `contender_6`, their comparison theorems, the old `maxBy` winner, or P5's named contender constants. It does reuse the structural ingredients `index`, `stage`, and `p5Family`; this is analogous to extending a hierarchy from its previous notation system rather than inserting the previous final numeral as a seed.

As with Contender 6, the inherited P5 seed family contains its historical `oldMaximum` guard as a function of depth. The package discloses this rather than presenting the dependency closure as cleaner than it is.

8 Growth analysis: formal facts versus heuristic calibration

8.1 What is formally established

The following statements require no ordinal analysis:

1. Contender 6 is strictly smaller than Contender 7.
2. The proof factors through forty-two guarded branch steps and one outer root step.
3. The level-41 autonomous payload is strictly smaller than Contender 7.
4. The definition is total and constructive in Lean.

These are the claims on which adoption as the next BusyLean contender can rest.

8.2 Intended ordinal reading

For an informal scale comparison, write α for the rank of the old self-generated index and L for the effective strength of one reflection step using the W-language. The repository's current heuristic describes Contender 6 as roughly a fast-growing value at an index of the form $L \cdot \alpha$.

Let H_0 denote the diagonal common-critical-point transformer generated by `autonomous`, and let H_{k+1} enumerate fixed points of H_k. Under the normal-function interpretation of the OTree constructors, the raw payload R_{41} has rank on the scale

$$H_{41}(\alpha + 1).$$

Consequently the new schedule has rank at least that large, plus the finite guard and root structure, and the reflected numerical value is heuristically on a scale resembling

$$F_{L \cdot H_{41}(\alpha+1)}(42),$$

whereas the old entry is on the scale $F_{L \cdot \alpha}(42)$.

This is a qualitative statement: repeated fixed-point enumeration above α should dwarf multiplication by α or any finite stack of ordinary reflection steps.

Warning 4. The expression above is not a theorem of the submitted Lean development. It depends on an external ordinal semantics for OTree, rank preservation of `encode`/`realize`, normality of the generated transformers, and a calibration of the object-language reflection operator. The package therefore uses words such as "intended", "heuristic", and "on the scale of" rather than attaching a false precision to a named proof-theoretic ordinal.

8.3 Why this still constitutes a significant advance

Even without the heuristic, the advance is not merely the definition `contender_6 + 1`. The new number performs fresh forest reflection over:

- the complete old self-generated index;
- forty-two recursively guarded extensions of that index;
- an explicit hierarchy of critical-point payloads built above the old index, culminating in the level-41 higher derivative;

- an outer root that exposes the whole branch family to the relativized W-language.

Thus the construction adds a new schedule language and a new layer of reflection, while preserving a short, auditable comparison proof.

9 Verification protocol

The archive includes the exact Lean sources, the workflow used for verification, and captured logs. The intended checks are:

1. cleanly build `BusyLean.Contender7`, which contains definitions only;
2. build the full library including `BusyLean.Comparison7`;
3. run `test/Contender7Audit.lean`;
4. print the axiom dependencies of the final definition and principal theorems;
5. assert that the definition's transitive dependency closure avoids the forbidden previous contender constants and winner selectors;
6. record the exact Lean and Lake versions.

The final verification run succeeded on GitHub Actions at repository commit

`c01efa921d20fcb1d731329ef5582fd16c933fcb`

using Lean 4.34.0 (kernel commit `293d5d0c0c3f3dded4688b3ccd6a33939ac5102b`) and Lake 5.0.0 on Ubuntu 24.04. With both Lake build directories removed, the definition phase took 1.45 seconds against a 15-second budget, and the remaining proof phase took 2.15 seconds against a 60-second budget. Seventeen declarations across the historical and new audits were checked; every reported axiom belonged to the allowlist {`propext`, `Quot.sound`}. The new definition and each principal Contender 7 theorem reported `propext` only. The transitive closure of `contender_7` contained 575 constants and avoided all eight forbidden previous-winner names. The feasible sanity check evaluated `contender7Family 0` to 4; the final input 42 was not evaluated.

The exact checker output and toolchain record are included in the archive and should be treated as authoritative over any prose in this paper.

10 Limitations and open obligations

1. **No formal ordinal quotient.** OTree is raw syntax. Equivalent fundamental sequences are not identified, and no well-order comparison decision procedure is claimed.
2. **No formal normality proof.** The fixed-point interpretation of `derivative`, `critical`, `autonomous`, and `hyper` is mathematically standard in shape, but normality and exact semantic equations are not mechanized here.
3. **No exact proof-theoretic calibration.** The fast-growing hierarchy labels are explanatory estimates, not kernel-checked bounds.
4. **Inherited seed complexity.** The proposal inherits P5's seed family and its historical guard. A future entry could replace this with a cleaner independently calibrated family.

5. **Finite visible level.** Input 42 exposes `hyper 41`. This is deliberate for a closed contest entry, but an autonomous diagonal over the entire higher-derivative sequence may offer a stronger next step.

These limitations do not affect the theorem $\mathsf{C}_6 < \mathsf{C}_7$ or the computability of C_7.

11 Further research questions

1. **Formal rank semantics.** Define a suitable ordinal or well-founded rank semantics for `OTree` and `WTree Nat`; prove that `realize` preserves rank and characterize `encode`.

2. **Normality and derivatives.** State a syntactic invariant ensuring that an OTree transformer represents a normal function, then prove that `omegaPow`, `critical`, `autonomous`, and `hyper` preserve it.

3. **Certified lower bounds.** Formalize a fast-growing hierarchy over OTree and prove that the numerical `stage` operation dominates selected hierarchy levels. This would turn the present ordinal heuristic into an explicit Lean lower-bound theorem.

4. **Autonomous diagonalization.** Replace the fixed index 41 by a self-generated schedule that diagonalizes over all higher derivatives while remaining structurally recursive and auditable.

5. **Stronger object languages.** Implement normalization and bounded maxima for System F, System F_ω, or a carefully controlled dependent calculus, then relativize those languages to the present family. A stronger base language should dominate any improvement that changes only the external tree schedule.

6. **Independent kernel checking.** Run Lean's independent checker or another trusted checker on the compiled environment, and incorporate an automated axiom allowlist audit in the repository CI.

7. **Definition-budget optimization.** Measure clean builds on the official contest hardware, reduce imports and generated code, and keep the definition/proof split within the repository's time budgets.

8. **Eliminate the historical guard.** Construct a seed family whose domination of earlier entries is proved through an independent inclusion theorem rather than an inherited maximum over the old language.

9. **General guard combinator.** Abstract the guarded-branch pattern and prove generic theorems for finite chains, payload visibility, and quantitative additive gaps at each reflection step.

10. **Semantic robustness under alternative fundamental sequences.** Determine which growth comparisons survive changes in the realization of successor and limit trees, and isolate the invariants actually needed by the reflection engine.

12 Integration notes

The archive is arranged so that its `lean/` directory can be overlaid on a checkout of BusyLean. The principal files are:

File	Purpose
`BusyLean/Contender7.lean`	definitions only
`BusyLean/Comparison7.lean`	strict comparisons and payload theorems
`test/Contender7Audit.lean`	axiom and dependency audit
`BusyLean.lean`	exports the new modules
`.github/workflows/contender7-ci.yml`	reproducible CI verification

The development branch used for this proposal is

`research/autonomous-reflection-c7-20261012`.

Adoption should proceed by reviewing the source and verification logs, not by trusting the informal size calibration.

13 Conclusion

Contender 7 combines a conservative proof strategy with an aggressive growth strategy. Conservatively, the old Contender 6 schedule is exposed as branch 0, and forty-two explicit guard nodes plus the outer root provide a transparent strict comparison. Aggressively, the same construction carries a hierarchy of Veblen-inspired critical-point payloads generated above the old self-referential index, culminating at the forty-first higher derivative visible at input 42.

The resulting Lean term is closed, structurally recursive, and computable in principle. Its essential comparison theorem is small enough to audit by hand, while the autonomous payload supplies a credible and extensible route far beyond the incumbent's schedule. The proposal therefore meets the practical standard for a new BusyLean contender while clearly separating formal theorem, engineering verification, and proof-theoretic heuristic.

A Principal theorem inventory

Declaration	Meaning
`contender7_branch_step_lt`	Every guarded branch is strictly below its successor at every input.
`contender7_raw_lt_next`	Payload R_k is strictly below guarded branch B_{k+1} whenever input is at least one.
`contender7_branch_zero_lt_succ`	The finite transitive closure from branch zero to branch $k+1$.
`finalFamily_lt_contender7Branch42`	Contender 6's family at input 42 is below branch 42 after forty-two guard steps.
`contender7Branch42_lt_contender_7`	The outer root strictly exceeds branch 42.
`contender_6_lt_contender_7`	Required incumbent comparison.
`contender_5_lt_contender_7`	Transitive comparison with the exact Contender 5 port.
`hyper41_lt_contender7Branch42`	The level-41 autonomous payload is present below branch 42.

Declaration	Meaning
`hyper41_lt_contender_7`	The final number strictly exceeds the level-41 payload.

B Reproduction commands

From a BusyLean checkout containing the patch:

```
lake build BusyLean.Contender7
lake build
lake env lean test/Contender7Audit.lean
```

Do not evaluate `contender_7`. The audit evaluates only a feasible input-zero sanity check for the family.

C Bibliographic note

The use of continuous increasing ordinal functions and fixed-point enumeration is inspired by Veblen's original hierarchy. The term "autonomous" is used in the broad proof-theoretic tradition associated with transfinite progressions and predicative analysis. The present construction is not claimed to reproduce any one historical notation system exactly.

References

[1] O. Veblen, *Continuous Increasing Functions of Finite and Transfinite Ordinals*, Transactions of the American Mathematical Society 9 (1908), 280–292. DOI: https://doi.org/10.1090/S0002-9947-1908-1500814-9.

[2] S. Feferman, *Transfinite Recursive Progressions of Axiomatic Theories*, Journal of Symbolic Logic 27 (1962), 259–316.

[3] S. Feferman, *Systems of Predicative Analysis*, Journal of Symbolic Logic 29 (1964), 1–30. DOI: https://doi.org/10.2307/2269764.

[4] K. Schuette, *Predicative Well-Orderings*, in Formal Systems and Recursive Functions, Studies in Logic and the Foundations of Mathematics 40 (1965), 280–303. DOI: https://doi.org/10.1016/S0049-237X(08)71694-X.

[5] The Lean Project, *Recursion - Recursive Functions in Lean 4*, https://lean4.dev/language/control-flow/recursion.

[6] V. Reshetnikov et al., *BusyLean repository*, https://github.com/VladimirReshetnikov/BusyLean.